AssetTrust v0.47

Security Implementation and Release Review

11 November 2026 | Core database relationship integrity | Internal development checkpoint

Release verdict: NO-GO - not approved for production

Executive summary

Version 0.47 adds database constraints that prevent assets, verification cases, evidence and documents from referencing parent records in a different organization. It also ensures that when a document refers to both a case and an asset, that asset belongs to the referenced case.

Important limitation: This is referential tenant integrity, not database-wide row-level security. The existing v0.46 RLS still covers three private PDF-layout tables only. Direct SQL read access to other private tables remains an open security issue.

New database safeguards

Protected record	New organization-aware constraint
Asset	Creator belongs to the asset organization
Verification case	Asset, creator and optional assignee belong to the case organization
Case evidence	Case, recorder and optional reviewer belong to the evidence organization
Document	Asset, case and uploader belong to document organization; case/asset pair must match

Migration safety

Migration **022_core_tenant_relationship_integrity.sql** adds four composite unique keys and eleven composite foreign keys. It validates existing rows within one transaction. Incorrect historical associations make the migration fail and roll back instead of being silently deleted, bypassed, or altered.

Test evidence

Validation	Result
New v0.47 offline tests	10 of 10 passed
Historical/current version suites	45 of 45 passed (v0.03 to v0.47)
CI static completeness	Passed: 45 suites and 21 declared PostgreSQL contracts
Architecture + TypeScript/TSX syntax	Passed: 102 source files (not a full build)
Migration source checksum manifest	22 of 22 SHA-256 fingerprints match
Current-source secret scan	1,796 files inspected; zero high-confidence matches
Unsafe PostgreSQL test-target refusal	Passed: rejected invalid disposable database
Live PostgreSQL migration and FK contract	NOT RUN: database and dependencies unavailable
Clean npm install / full app build	BLOCKED / FAILED: no genuine lockfile or dependencies
Full API tests	BLOCKED / FAILED: missing project dependencies
Production release preflight	BLOCKED: 3 of 11 prerequisites present

What the results prove - and do not prove

The passing results verify source-level contracts, deterministic tests, migration-file hashes and CI wiring in this environment. They do not prove that PostgreSQL actually enforced the constraints, that browser sessions are safe, or that two tenants are isolated in a running deployment.

The archive lacks the original Git history. Therefore, the snapshot secret scan cannot replace a full historical repository scan or rotation of compromised credentials.

Next required release gates

1. Back up and inspect

Restore a verified backup to an isolated, disposable PostgreSQL database; run the read-only core integrity audit and investigate all non-zero counts.

2. Install and build

Generate and review a genuine npm lockfile, run npm ci, all tests, both application builds, dependency vulnerability audit and SBOM checks.

3. Prove the live SQL constraints

Apply migration 022 on the disposable database. Run the guarded two-tenant foreign-key contract and legitimate asset/case/document workflows.

4. Extend true row-level security

Scope every core query and background worker transaction correctly before enabling FORCE RLS for the remaining private tables. Verify non-owner runtime roles and pooled connection reuse.

5. Finish release security

Prove HTTPS/HSTS, secure sessions/CSRF, admin MFA, rate limits, upload scanning, SSH/firewall restrictions, key custody, backups, history secret scanning and penetration tests.

Deployment decision

Do not process real customer evidence or accept paying customers until live PostgreSQL integration, critical security controls and independent review are completed. Version 0.47 is an **offline-tested development checkpoint only**.

Operator references: V47_KALI_VALIDATION.md, V47_HANDOVER.md, TEST_EVIDENCE_V47.md and tools/sql/v47-core-integrity-audit.sql.